complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + collect() accumulate elements

filter() null checks + collect() accumulate elements

filter() comparisons + collect() accumulate elements

filter() method calls returning boolean + collect() accumulate elements

distinct() rely on equals() / hashcode() + collect() accumulate elements

distinct() immutable objects + collect() accumulate elements

map() transform objects + collect() accumulate elements

map() create new objects + collect() accumulate elements

map() call pure functions + collect() accumulate elements

flatmap() return stream + collect() accumulate elements

flatmap() split data + collect() accumulate elements

flatmap() expand collections + collect() accumulate elements

sorted() natural ordering + collect() accumulate elements

sorted() custom comparator + collect() accumulate elements

sorted() multi-field compare + collect() accumulate elements

peek() logging + collect() accumulate elements

peek() debugging + collect() accumulate elements

peek() metrics + collect() accumulate elements

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, self-contained Java class examples** for **every requested Stream combination**, each focusing on **one intermediate operation + collect()**.

---

1 filter()— boolean checks + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterBooleanCheckExample {
   public static void main(String[] args) {
      List < Integer > numbers = List.of(10, 15, 20, 25, 30);

      List < Integer > evenNumbers = numbers.stream()
         .filter(n - > n % 2 == 0)                                    // boolean check
         .collect(Collectors.toList());

      System.out.println(evenNumbers);
   }
}
```

---

2 filter()— null checks + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterNullCheckExample {
   public static void main(String[] args) {
      List < String > names = Arrays.asList("Anurag", null, "Bharti", null);

      List < String > nonNullNames = names.stream()
         .filter(Objects::nonNull)
         .collect(Collectors.toList());

      System.out.println(nonNullNames);
   }
}
```

---

3 filter()— comparisons + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterComparisonExample {
   public static void main(String[] args) {
      List < Integer > salaries = List.of(20000, 35000, 50000);

      List < Integer > highSalaries = salaries.stream()
         .filter(s - > s > 30000)
         .collect(Collectors.toList());

      System.out.println(highSalaries);
   }
}
```

---

4 filter()— method calls returning boolean + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterMethodCallExample {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List < User > users = List.of(
         new User("Anurag", true),
         new User("Ravi", false)
      );

      List < User > activeUsers = users.stream()
         .filter(User::isActive)
         .collect(Collectors.toList());

      System.out.println(activeUsers.size());
   }
}
```

---

5 distinct()— rely on equals() / hashCode() + collect()

```
import java.util.*;
import java.util.stream.*;

public class DistinctWithEqualsExample {

   static class Person {
      String name;

      Person(String name) {
         this.name = name;
      }

      @Override
      public boolean equals(Object o) {
         return o instanceof Person p && name.equals(p.name);
      }

      @Override
      public int hashCode() {
         return Objects.hash(name);
      }
   }

   public static void main(String[] args) {
      List < Person > people = List.of(
         new Person("Anurag"),
         new Person("Anurag"),
         new Person("Ravi")
      );

      List < Person > unique = people.stream()
         .distinct()
         .collect(Collectors.toList());

      System.out.println(unique.size());
   }
}
```

---

6 distinct()— immutable objects + collect()

```
import java.util.*;
import java.util.stream.*;

public class DistinctImmutableExample {
   public static void main(String[] args) {
      List < String > values = List.of("A", "B", "A", "C");

      List < String > unique = values.stream()
         .distinct()
         .collect(Collectors.toList());

      System.out.println(unique);
   }
}
```

---

7 map()— transform objects + collect()

```
import java.util.*;
import java.util.stream.*;

public class MapTransformExample {
   public static void main(String[] args) {
      List < String > names = List.of("anurag", "bharti");

      List < String > upper = names.stream()
         .map(String::toUpperCase)
         .collect(Collectors.toList());

      System.out.println(upper);
   }
}
```

---

8 map()— create new objects + collect()

```
import java.util.*;
import java.util.stream.*;

public class MapCreateObjectExample {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {
      List < String > names = List.of("Anurag", "Ravi");

      List < UserDTO > dtos = names.stream()
         .map(UserDTO::new)
         .collect(Collectors.toList());

      System.out.println(dtos.size());
   }
}
```

---

9 map()— call pure functions + collect()

```
import java.util.*;
import java.util.stream.*;

public class MapPureFunctionExample {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List < Integer > numbers = List.of(2, 3, 4);

      List < Integer > squares = numbers.stream()
         .map(MapPureFunctionExample::square)
         .collect(Collectors.toList());

      System.out.println(squares);
   }
```

```
}
```

---

10 flatMap() — return Stream + collect()

```
import java.util.*;
import java.util.stream.*;

public class FlatMapStreamExample {
   public static void main(String[] args) {
      List < List < Integer >> lists = List.of(
         List.of(1, 2),
         List.of(3, 4)
      );

      List < Integer > result = lists.stream()
         .flatMap(List::stream)
         .collect(Collectors.toList());

      System.out.println(result);
   }
}
```

---

11 flatMap()— split data + collect()

```
import java.util.*;
import java.util.stream.*;

public class FlatMapSplitExample {
   public static void main(String[] args) {
      List < String > sentences = List.of("Java Stream", "Flat Map");

      List < String > words = sentences.stream()
         .flatMap(s - > Arrays.stream(s.split(" ")))
         .collect(Collectors.toList());

      System.out.println(words);
   }
}
```

---

12 flatMap()— expand collections + collect()

```
import java.util.*;
import java.util.stream.*;

public class FlatMapExpandExample {
   public static void main(String[] args) {
      Map < String, List < String >> data = Map.of(
         "A", List.of("1", "2"),
         "B", List.of("3")
      );

      List < String > values = data.values().stream()
         .flatMap(Collection::stream)
         .collect(Collectors.toList());
      System.out.println(values);
   }
}
```

---

13 sorted()— natural ordering + collect()

```
import java.util.*;
import java.util.stream.*;

public class SortedNaturalExample {
   public static void main(String[] args) {
      List < Integer > numbers = List.of(5, 1, 3);

      List < Integer > sorted = numbers.stream()
         .sorted()
         .collect(Collectors.toList());

      System.out.println(sorted);
   }
}
```

---

14 sorted()— custom comparator + collect()

```
import java.util.*;
import java.util.stream.*;

public class SortedCustomComparatorExample {
   public static void main(String[] args) {
      List < String > names = List.of("Anurag", "Ravi", "Alok");

      List < String > sorted = names.stream()
         .sorted(Comparator.reverseOrder())
         .collect(Collectors.toList());

      System.out.println(sorted);
   }
}
```

---

15 sorted()— multi - field compare + collect()

```
import java.util.*;
import java.util.stream.*;

public class SortedMultiFieldExample {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
      List < Employee > employees = List.of(
         new Employee("Anurag", 30),
         new Employee("Anurag", 25),
         new Employee("Ravi", 35)
      );

      List < Employee > sorted = employees.stream()
         .sorted(Comparator
            .comparing((Employee e) - > e.name)
            .thenComparingInt(e - > e.age))
         .collect(Collectors.toList());

      System.out.println(sorted.size());
   }
}
```

---

16 peek()— logging + collect()

```
import java.util.*;
import java.util.stream.*;

public class PeekLoggingExample {
   public static void main(String[] args) {
      List < Integer > result = List.of(1, 2, 3).stream()
         .peek(n - > System.out.println("Processing: " + n))
         .collect(Collectors.toList());

      System.out.println(result);
```

```
    }
}
```

---

17 peek()— debugging + collect()

```
import java.util.*;
import java.util.stream.*;

public class PeekDebugExample {
    public static void main(String[] args) {
        List < Integer > result = List.of(10, 20, 30).stream()
            .peek(n - > {
                if (n == 20) System.out.println("Breakpoint value: " + n);
            })
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

---

18 peek()— metrics + collect()

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.*;

public class PeekMetricsExample {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        List < Integer > result = List.of(5, 10, 15).stream()
            .peek(n - > counter.incrementAndGet())
            .collect(Collectors.toList());

        System.out.println("Processed count: " + counter.get());
    }
}
```